Exercițiul 2 — Observer: un eveniment, N reacții

Proiect `design-patterns-java` · pachetul `app.ex2`

Context

Aceeași comandă din magazin, dar acum urmărim **starea** ei: `Plasata → Expediata → Livrata`. La FIECARE schimbare de stare reacționează mai multe părți: clientul primește un email, se scrie o linie în jurnalul de livrări, depozitul își pregătește pasul următor.

Prima variantă: `schimbaStare` cheamă direct `email.trimite(...)`, `jurnal.scrie(...)`, `depozit.pregateste(...)`. `Comanda` ajunge să cunoască fiecare parte pe nume, iar o notificare push în plus înseamnă să redeschizi `schimbaStare`. La `ex1` se schimba **cum se calculează**; aici se schimbă **cine reacționează**.

Observer e răspunsul: `Comanda` ține o listă de observatori care au semnat același contract și, la fiecare schimbare, îi anunță pe toți la fel, fără să știe cine sunt. Teoria e în `TEORIE.pdf`, lecția 2.

Contractul

```
public interface Observator {
    void actualizeaza(String stareNoua);
}
```

Ce afișează fiecare observator

Observator	La `actualizeaza("X")` afișează
`NotificatorEmail`	`[EMAIL] Comanda a trecut in starea: X`
`JurnalLivrare`	`[LOG] Stare inregistrata: X`
`PanouDepozit`	`[DEPOZIT] Pregatesc pentru: X`

Pașii (în ordinea asta — primul e capcana)

1. **Simte durerea întâi.** Pe ciornă, scrie `schimbaStare` cu apeluri directe către cele trei părți. Răspunde: câte locuri atingi când adaugi o notificare push? Ce cunoaște `Comanda` și n-ar trebui?
2. **Scrie observatorii.** Cele 3 clase implementează `Observator`, fiecare cu afișarea ei din tabel.
3. **Abonarea.** `aboneaza` adaugă observatorul în lista `observatori`, `dezaboneaza` îl scoate.
4. **Fă `Comanda` să anunțe, nu să știe.** `schimbaStare(stareNoua)`, în ordinea asta:
   - dacă `stareNoua` e `null` sau goală → `IllegalArgumentException("State cannot be empty")`;
   - dacă e egală cu starea curentă → nu face nimic (nu anunță degeaba);
   - altfel setează `stare` și parcurge lista cu `for`, chemând `actualizeaza(stareNoua)` pe fiecare observator.

Ce găsești deja în proiect

Fișier	Ce conține
`src/app/Main.java`	cheamă `Testare1` … `Testare6`; nu-l modifici
`src/app/ex2/Testare2.java`	`public static void ruleaza()` gol — îl scrii tu
`models/Observator.java`	contractul de mai sus, gata
`NotificatorEmail`, `JurnalLivrare`, `PanouDepozit`	`actualizeaza` aruncă `UnsupportedOperationException`

`Comanda`	câmpurile `client`, `stare` (pornește din `"Plasata"`) și lista `observatori`; constructorul cu validare, `getClient()`, `getStare()` gata; restul aruncă `UnsupportedOperationException`

Semnăturile pe care le completezi în `Comanda`:

```
public void aboneaza(Observator observator)
public void dezaboneaza(Observator observator)
public void schimbaStare(String stareNoua)
```

Cerințe

1. Cele 3 observatoare, cu afișările din tabel.
2. În `Comanda`: `aboneaza`, `dezaboneaza` și `schimbaStare`, ca la pașii 3–4.
3. `Testare2.ruleaza()` exact așa:

```
public static void ruleaza() {
    Comanda comanda = new Comanda("Ana");
    PanouDepozit depozit = new PanouDepozit();
    comanda.aboneaza(new NotificatorEmail());
    comanda.aboneaza(new JurnalLivrare());
    comanda.aboneaza(depozit);
    comanda.schimbaStare("Expediata");
    comanda.schimbaStare("Expediata");
    comanda.dezaboneaza(depozit);
    comanda.schimbaStare("Livrata");
    System.out.println("Stare finala: " + comanda.getStare());
    try {
        comanda.schimbaStare(" ");
    } catch (IllegalArgumentException ex) {
        System.out.println(ex.getMessage());
    }
}
```

Ieșirea așteptată la **Run 'Main.main()'**, după cele 4 rânduri de la `ex1`, exact așa:

```
[EMAIL] Comanda a trecut in starea: Expediata
[LOG] Stare inregistrata: Expediata
[DEPOZIT] Pregatesc pentru: Expediata
[EMAIL] Comanda a trecut in starea: Livrata
[LOG] Stare inregistrata: Livrata
Stare finala: Livrata
State cannot be empty
```

Citește-o: al doilea `"Expediata"` nu produce nimic (starea nu s-a schimbat), iar la `"Livrata"` depozitul tace, pentru că s-a dezabonat.

Constrângeri

- `Comanda` NU cunoaște nicio clasă concretă de observator — doar `Observator`.
- Fără `instanceof`, `getClass()`. Fără `Map`.
- Mesajele de eroare în engleză.

Bonus

- Abonează un al patrulea observator **după** `"Expediata"`. Ce vede el? Un observator prinde doar ce se întâmplă după ce s-a abonat.
- Întrebare de gândit, legătura cu `ex1`: la Strategy, `Comanda` avea UN colaborator și îl ÎNTREBA („cât costă?”). La Observer are MULȚI și îi ANUNȚĂ. De ce `actualizeaza` întoarce `void`?